

Tap Light — Circuit Walkthrough

What this document is: a component-by-component explanation of the minimal Tap Light breadboard — why every piece is there and what job it does. It pairs the physical circuit with `firmware/tap_light/tap_light.ino` so the hardware and the code explain each other.

The one-sentence summary: the hardware turns a mechanical tap into a small, *safe* positive voltage bump that a 3.3 V ADC pin can read, and it turns a logic pin into a clean data stream for an LED strip; the firmware decides "was that a real tap?" and advances the color mode.

The signal chain at a glance

There are two independent paths sharing one board and one ground:

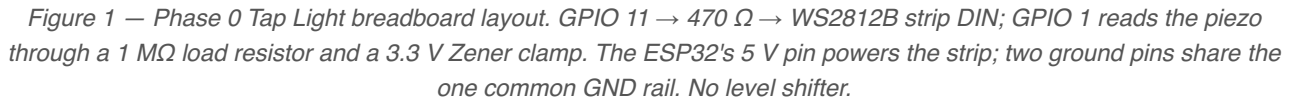
Input (sensing) — left side of the board

tap → piezo makes a ringing $\pm$ spike → a 3.3 V Zener clamps it safe and a 1 M Ω resistor to ground sets a stable resting level → GPIO 1 (ADC) samples it → firmware sees a positive jump above its baseline

Output (display) — right side of the board

GPIO 11 → data series resistor → strip DIN → 30x WS2812B light up; the onboard RGB pixel mirrors the same color

Everything else on the board — the big capacitor, the ground jumpers — exists to keep those two paths clean and referenced to a **common ground**.



1. The ESP32-S3 board + USB power

- **GPIO 1** — an **ADC1** pin, set up as a high-impedance analog input (`pinMode(PIEZ0_PIN, INPUT)` , `tap_light.ino:71`). This reads the piezo.
- **GPIO 11** — the WS2812B strip data line (`LED_PIN` , `tap_light.ino:7`).
- **GPIO 38** — the onboard RGB "NeoPixel" (your board is silk-screened `RGB@I038`). The code also drives GPIO 48 to cover a second board revision (`tap_light.ino:14–15`), but only 38 is real on yours.

Octagonal Turn Counter — Tap Light Circuit Walkthrough • page 2 of 6

2. The piezo disc — the sensor

The brass disc is a **piezoelectric transducer**. Bending/stressing its ceramic layer (a tap does this) generates an electric charge across its faces. Electrically it behaves like **a small capacitor in series with a voltage source**, with two awkward properties:

- **It's AC by nature.** A tap isn't a clean step — it's a fast *ringing* spike that swings **both positive and negative** as the disc vibrates and settles.
- **The voltage can be huge.** An unloaded piezo can spike to **tens of volts** on a sharp tap. The ADC pin only tolerates roughly -0.3 V to $+3.6\text{ V}$. Wired straight in, a hard tap would slam the pin's internal protection diodes.

So the raw signal is too big, bipolar, and has no defined resting point. A resistor and a Zener diode fix all three problems.

3. Load / bleed resistor — *across the piezo, to ground*

The most important passive part (classically $\sim 1\text{ M}\Omega$ — this is the same resistor as the stock Arduino "Knock" example). Connected **across the piezo to ground**, it does three things at once:

1. **Gives the piezo a DC return path to ground.** The piezo is a capacitor; with nowhere to drain, charge accumulates and the reading drifts or latches. The resistor pulls the node back to a defined **resting voltage** (near 0 V) between taps — this is the level your firmware baseline locks onto.
2. **Sets the decay time of the spike** ($\tau = R \times C_{\text{piezo}}$). Too small and the spike is too brief to catch; $\sim 1\text{ M}\Omega$ stretches it enough to sample reliably at the firmware's 5 ms loop rate.
3. **Provides the ADC's ground reference.** Break this and GPIO 1 floats — the documented "constant taps regardless of the piezo, rail-to-rail readings" symptom. That's a *lost ground*, not a bad threshold.

4. Clamp — *a 3.3 V Zener from the sense node to ground*

The Zener is the protection element. A Zener diode conducts *backwards* the instant the voltage across it exceeds its rated value, so a **3.3 V Zener** from the sense node to ground does nothing during small taps — but the moment a hard tap tries to shove the node toward $+15\text{ V}$, the Zener turns on and **clamps it to $\sim 3.3\text{ V}$** , dumping the excess to ground before it reaches the pin. The piezo is a high-impedance source (a tiny capacitor), so it simply can't push enough current to overwhelm the clamp. The pin sees a safe positive bump capped right at the ADC's full scale; the negative half of the ring is clamped near ground the same way.

Resistor + Zener, two jobs: the $1\text{ M}\Omega$ *across* the piezo defines the resting/return level and the decay time; the 3.3 V Zener *across* the piezo caps the spike at the ADC's ceiling. Together they turn a wild $\pm$ tens-of-volts source into a tame $0\text{--}3.3\text{ V}$ signal — and because the Zener clamps directly, no series current-limiting resistor is needed.

Output side — how a number becomes light

5. GPIO 11 → data-line series resistor → strip

The green wire carries the WS2812B data signal out of GPIO 11. The resistor it passes through on the way to the strip's DIN is a **data-line series resistor** (~330–470 Ω) — standard practice for addressable LEDs. It **damps ringing and reflections** on the fast data edges and limits current into the first pixel's input, protecting that pixel and cleaning up the signal so the whole strip latches reliably. Skip it and you get occasional flicker or a flaky first LED, especially with longer data leads.

6. Strip power + the common-ground tie

The strip's **+5 V and GND come from an external supply** (the red/black leads leaving frame), not from the signal pins — 30 WS2812B pixels can pull well over an amp at full white, far more than a GPIO can source. The critical detail is the **extra ground you added**: the strip's supply ground and the ESP32's ground **must be tied together**. The data line is a *voltage* referenced to ground; if the strip and the controller don't share a ground, "high" and "low" have no common meaning and the strip misreads or ignores the data. Common ground is not optional — it's what makes the single data wire work.

7. The onboard RGB pixel (GPIO 38) — a mirror

Not a stand-in — with the strip live, the onboard pixel simply **mirrors the current mode** so there's a status light right on the board (`tap_light.ino:54`). One quirk: a single pixel can't render the rainbow *gradient* that mode 4 paints across the 30-LED strip (`tap_light.ino:45–49`), so instead the pixel **animates its hue over time** from `loop()` (`tap_light.ino:88–95`) — one LED cycling colors "reads" as rainbow.

Shared infrastructure

8. The big electrolytic capacitor — bulk supply reservoir

The large blue cap sits **across the power rails** and is a **bulk reservoir / decoupling capacitor** (the classic Adafruit NeoPixel recommendation is ~1000 μF). Two reasons it earns its place:

- **Addressable LEDs draw current in sharp pulses.** Driving 30 WS2812B pixels yanks current in fast bursts; the cap is a local energy reservoir that absorbs those bursts so the rail doesn't sag and brown out the chip mid-update.
- **A stable rail = a stable ADC.** The ESP32's ADC measures against a supply-derived reference, so rail ripple becomes noise on the piezo reading. Smoothing the supply keeps the baseline quiet — which is why tiny drift is all the firmware ever has to chase (`tap_light.ino:108`).

Polarity matters: an electrolytic is polarized. The striped leg is negative and goes to GND; the other to +V. It belongs across the *power rails* — never across the piezo, where a cap that big would swallow the tap spike entirely.

9. The jumper wires

Unglamorous but load-bearing:

- **3V3** → **red rail**, **GND** → **blue rail**: bring board power to the breadboard.
- **GND** → **piezo node**: the return path for the load resistor (§3).
- **Piezo node** → **GPIO 1**: the conditioned tap signal into the ADC.
- **GPIO 11** → **data resistor** → **strip DIN**: the display data path.
- **Strip supply GND** ↔ **board GND**: the common-ground tie (§6).

Where the hardware and firmware meet

The hardware deliberately hands the firmware a *messy* signal, and the firmware is built around that mess:

- **Adaptive baseline** (`tap_light.ino:74–79` , `108`): the resting voltage set by the resistors isn't fixed — it drifts with temperature, mounting pressure, and how the piezo is bent. The code averages the resting level at boot, then slowly tracks it, while **never letting a spike feed the average**. That's how repeated taps don't desensitize the sensor.
- **TAP_DELTA = 1000** (`tap_light.ino:24`): a spike must clear the baseline by 1000 counts (≈ 0.8 V on the 12-bit, 0–4095 ADC) — big enough to ignore rail noise, small enough that a normal tap clears it.
- **DEBOUNCE_MS = 500** (`tap_light.ino:25` , `100`): the piezo *rings*, so one physical tap is several electrical spikes. Debounce means "one tap = one mode change" by ignoring everything for 500 ms after the first spike.

Put together: **piezo makes the ring** → **the 3.3 V Zener clamps it safe** → **the 1 M Ω to ground gives it a stable resting level** → **the cap keeps the rail quiet** → **firmware watches for a positive jump above a self-adjusting baseline, once per half-second** → **it advances the mode** → **GPIO 11 drives the strip and the onboard pixel mirrors it**.

Component summary

Component	Where	Job	Necessary because
ESP32-S3 + USB	center	Reads ADC, drives data pins, runs firmware	It's the controller
Piezo disc	left, off-board	Converts a tap to a voltage spike	The sensor itself
Load resistor (~1 M Ω)	across piezo → GND	Defines resting level, sets decay, gives ground ref	Without it the ADC pin floats → chaotic reads
3.3 V Zener	sense node → GND	Clamps each tap to ~3.3 V	Keeps the tens-of-volts spike inside the pin's safe range

Component	Where	Job	Necessary because
Data resistor (~330–470 Ω)	GPIO 11 → strip DIN	Damps data-line ringing, protects pixel 1	Reliable strip latching, no flicker
Bulk cap (~1000 μ F)	across power rails	Reservoir for LED current bursts; steadies ADC ref	Prevents brownouts + baseline noise
Common-ground tie	strip supply ↔ board GND	Shared reference for the data signal	Data "high/low" is meaningless without it
Onboard RGB (GPIO 38)	on-board	Mirrors current mode as a status light	Convenience, not required for the strip

One honest caveat

Each component's *role* above is confident (from the code plus the layout), but component bands can't be read reliably from a photo — so which physical part is the 1 M Ω load, the 3.3 V Zener clamp, and the 470 Ω data resistor is inferred from position, not measured. To confirm the exact topology: read out each resistor's bands and how its legs connect, or watch the live baseline value the firmware already logs at boot (`tap_light.ino:80`) to see where the resting point actually sits.